

Hardening the Software Supply Chain

and Enforcing Least Privilege Across 20+ Integrations

Sigstore / cosign · SBOM · in-toto Provenance · SLSA Trusted Builds
Deployment Policy Enforcement · OCI IAM · Immutable Audit Trails

A project explanation written in plain English

This document explains the project from the ground up. It assumes you know nothing about software supply chain security. Every technical term is explained the first time it appears.

It contains four things: (1) a simple overall summary, (2) a detailed explanation of every component, (3) a detailed end-to-end workflow, and (4) two full-page diagrams with a walkthrough of every box and arrow.

Technical Design & Explanation Document

Contents

#	Section	What it gives you
1.	The Project in Simple Terms	The whole idea in one page, plus an everyday analogy
2.	Words You Need to Know	A plain-English dictionary for every term used
3.	The Components, One by One	All twelve building blocks explained end to end
4.	The End-to-End Workflow	All 16 steps, from a developer's commit to a running container
5.	The Architecture Diagram Explained	Full-page diagram with every zone, box and arrow described
6.	Which Attack Each Piece Stops	A threat-to-defence mapping table
7.	Explaining This Project in One Minute	A short spoken summary you can use directly

1. The Project in Simple Terms

1.1 The one-sentence version

In one sentence

This project makes sure that the only software allowed to run in production is software we can mathematically prove we built ourselves — from source code we reviewed, on machines we control — and it makes sure every person and every machine involved holds the smallest possible set of permissions, with a permanent, unchangeable record of everything that happened.

1.2 The everyday analogy

Think about a bottle of medicine on a pharmacy shelf. Before you swallow anything, four things reassure you:

1. There is a tamper-proof seal. If someone opened the bottle after it left the factory, the seal is broken and you can see it.
2. There is an ingredients label listing exactly what is inside, in what quantity.
3. There is a batch number that traces back to the exact factory, production line, and date it was made.
4. The pharmacy will not sell you a bottle that fails any of those checks. Nobody is allowed to say "it is probably fine, put it on the shelf anyway."

This project builds all four of those things for software:

Medicine bottle	This project	The component that does it
Tamper-proof seal	A cryptographic signature on the container image	Sigstore / cosign
Ingredients label	A list of every software package inside the image	SBOM
Batch number / factory record	A signed record of which source code and which build machine produced it	in-toto provenance
The pharmacy refusing a bad bottle	An automatic gate that blocks any deployment failing a check	Deployment policy enforcement
Only trained staff can touch stock	Every person and machine gets only the access it truly needs	OCI IAM, dynamic groups, instance principals
CCTV that cannot be erased	A permanent record of every action taken	Immutable audit trails

1.3 The problem this solves

Modern applications are not written from scratch. A typical container image is perhaps 5% code your team wrote and 95% code pulled in from the internet: base operating system images, open-source libraries, build tools, and plugins. It is assembled by automated build servers and shipped through registries and pipelines.

That creates a long chain of places where an attacker can insert something harmful without ever touching your source code:

- They compromise an open-source library you depend on, so malicious code arrives as a normal update.
- They compromise the build server, so the code that goes in is fine but the image that comes out is not.
- They steal registry credentials and push a poisoned image under a familiar name or tag.
- They take an image that was legitimately built for a test environment and push it into production.
- They use one over-permissive service account to move sideways from a low-value system into a critical one.

The traditional answer was to trust the pipeline: if the image arrived from our registry, it must be ours. This project replaces trust with proof. Every artifact carries evidence of its own origin, and every stage independently re-checks that evidence instead of taking the previous stage's word for it.

1.4 What was actually delivered

- Every container image is cryptographically signed at build time, and the signature is verified again before deployment.
- Every image ships with an SBOM — a complete, machine-readable ingredient list of every package inside it.
- Every image ships with in-toto provenance — a signed birth certificate naming the source commit, the builder, the build parameters, and the exact output.
- Builds run on hardened, isolated, scripted builders aligned to the SLSA framework, so no human can quietly alter a build in progress.
- A deployment policy gate blocks anything that is unsigned, unlogged, built by an unapproved builder, or carrying a banned vulnerability or licence.
- Access across 20+ integrated systems is scoped down to least privilege using OCI IAM compartments, dynamic groups, instance principals, and narrowly written policies.
- Long-lived static credentials were removed from the pipeline in favour of short-lived, automatically issued machine identities.
- Every signature, decision, and API call lands in an append-only audit trail that nobody — including administrators — can quietly edit.

1.5 The two halves of the project

The single long project description is really two connected pieces of work. It helps to keep them separate in your head.

	Half A — Supply chain hardening	Half B — Least privilege access
Question it answers	Can I prove this software is what I think it is?	Should this identity be allowed to do this action?
Protects against	Tampered, forged, or unknown artifacts	Stolen credentials and sideways movement between systems
Main tools	cosign, SBOM, in-toto, SLSA, policy gate	OCI compartments, dynamic groups, instance principals, policies
Proof it produces	Signatures, attestations, transparency log entries	Audit records of every allowed and denied call
Where it acts	Build time and deploy time	Continuously, on every API call

The two halves reinforce each other. Signing is only meaningful if an attacker cannot steal the signing identity, which is Half B's job. Least privilege is only meaningful if the software holding those privileges is trustworthy, which is Half A's job.

2. Words You Need to Know

Read this table once and the rest of the document will be straightforward. Nothing here assumes prior security knowledge.

Term	What it actually means
Artifact	Any finished thing the build produces — most often a container image, but also a .jar, .zip, or binary.
Container image	A packaged, ready-to-run copy of an application plus everything it needs to run.
Digest (sha256:...)	A fingerprint calculated from the file's contents. Change one byte and the fingerprint changes completely. Two identical files always produce the same digest.
Tag (e.g. :latest)	A friendly nickname pointing at an image. Nicknames can be moved to point somewhere else, which is why this project trusts digests, not tags.
Signing	Using a private key to produce a small proof that a specific digest was approved by a specific identity.
Verification	Re-checking that proof with the matching public key, to confirm both who signed it and that nothing has changed since.
SBOM	Software Bill of Materials. A structured list of every package and version inside an artifact.
Attestation	A signed statement about an artifact. "This SBOM belongs to this digest" is an attestation.
Provenance	The origin story of an artifact: which source, which builder, which parameters, which time.
Registry (OCIR)	The storage service that holds container images. OCIR is Oracle Cloud Infrastructure Registry.
Admission / policy gate	A checkpoint that inspects a deployment request and decides admit or block, before anything starts running.
Least privilege	Give every identity the minimum permissions needed for its job, and nothing more.
Compartment	In Oracle Cloud, a walled-off area holding resources. Permissions are granted inside one compartment at a time.
Principal	Any identity that can make a request — a person, a group, a service, or a machine.
Immutable	Write once, then unchangeable. Records can be added and read, never edited or deleted.
Fail-closed	When something cannot be verified, the safe default is to refuse rather than allow.

3. The Components, One by One

Each component below is explained in the same five-part shape: what it is, why it matters, how it works end to end, what it looks like in practice, and what would break without it.

3.1 Sigstore and cosign — signing and verifying images

What it is: Sigstore is a free, open toolkit for signing software. cosign is the command-line tool within it that signs and verifies container images. Together they answer one question: who approved this exact artifact?

Why it matters: A registry can be breached and an image replaced. Without a signature there is nothing to distinguish the real image from an impostor, because both sit under the same name in the same trusted-looking place. A signature makes forgery detectable.

How it works, end to end

1. The build finishes and produces a container image with a unique digest, for example sha256:9f2c... — a fingerprint of the image's exact contents.
2. cosign runs inside the build job. Instead of reading a stored private key, it asks the CI system for an identity token: a short-lived, signed statement saying "this is build #4471 of repository payments-api, running the release workflow."
3. cosign sends that token to Fulcio, Sigstore's certificate authority. Fulcio validates the token and issues a certificate that is valid for roughly ten minutes and that permanently records which identity and which workflow it was issued to.
4. cosign generates a signing key in memory, signs the image digest with it, and never writes the key to disk.
5. cosign publishes the signature, along with the certificate, into Rekor — a public, append-only transparency log. Rekor timestamps the entry and returns a proof that it was included.
6. The certificate expires. The in-memory key is discarded. There is now no signing secret anywhere for an attacker to steal.
7. Later, at deployment time, cosign verify runs in reverse: it fetches the signature, confirms the mathematics, checks that the certificate identity matches an approved workflow, and confirms the entry genuinely exists in Rekor.

In practice

This approach is called keyless signing. The identity being trusted is not a secret file but the build workflow itself. If someone tries to sign an image from their laptop, the certificate will name their personal identity rather than the approved pipeline, and verification will reject it.

Signatures are always bound to the digest, never to a tag. Repointing the tag latest at a malicious image does not carry the signature across, so the substitution is caught.

If this were missing

Anyone who obtained registry write access could publish an image under a trusted name, and no downstream system would have any way to tell the difference.

3.2 SBOM generation — the ingredient list

What it is: A Software Bill of Materials is a complete, machine-readable inventory of everything inside an artifact: every library, its exact version, its licence, and where it came from. Standard formats are CycloneDX and SPDX.

Why it matters: When a serious vulnerability is announced in a widely used library, the urgent question is: are we affected, and where? Without an SBOM, answering that means manually rebuilding and inspecting hundreds of images over days. With an SBOM, it is a query answered in seconds.

How it works, end to end

1. Immediately after the image is built, an SBOM generator scans the finished image — not just the dependency manifest, because the manifest describes what was requested while the image contains what actually arrived.
2. It walks every layer of the image and inventories operating-system packages, language packages, and pulled-in transitive dependencies that no developer explicitly requested.
3. For each entry it records name, exact version, licence, and a package identifier that uniquely names that component in the wider ecosystem.
4. The result is written as a structured file, typically a few hundred kilobytes of JSON.
5. cosign attaches the SBOM to the image as a signed attestation, cryptographically bound to the same image digest. The ingredient list cannot be swapped for a friendlier one.
6. The SBOM is scanned against vulnerability databases and against the organisation's licence rules, and both the SBOM and the scan result are archived.
7. At deployment, the policy gate reads the SBOM again and re-applies current rules — because a component that was clean when built may have had a critical vulnerability disclosed since.

In practice

The SBOM is what turns "we think we might use that library somewhere" into "it is in these 14 images, at these versions, and 3 of them are running in production right now." It also catches licence problems before they reach a customer, such as a copyleft-licensed dependency arriving inside a commercially distributed product.

If this were missing

Vulnerability response becomes guesswork and takes days instead of minutes, and unwanted licences can enter a product unnoticed.

3.3 in-toto build provenance — the birth certificate

What it is: in-toto is a framework for recording and verifying how software was produced. The provenance document it generates states, in a standard signed format, exactly what went into a build and exactly what came out.

Why it matters: A signature proves the artifact has not changed since signing. It does not prove the artifact was built from the right source, on the right machine, in the right way. Provenance answers those questions.

What the provenance document records

- Subject: the digest of the artifact this statement is about.
- Source: the repository URL and the exact commit hash the build started from.
- Builder: the identity of the build system that produced it.
- Build definition: the workflow file, entry point, and parameters used.
- Materials: the dependencies and base images pulled in, with their digests.
- Timestamps: when the build started and finished.

How it works, end to end

1. As the build starts, the builder records the source repository, the commit hash, and the workflow parameters it was invoked with.
2. During the build it records every external input it fetched, including base image digests and dependency digests.
3. When the build finishes, it records the digest of the output artifact.
4. All of this is assembled into a standard in-toto statement — a small JSON document with a defined schema.
5. cosign signs that statement using the same keyless flow as the image itself, so the provenance carries the builder's verified identity.
6. The signed statement is attached to the image as a provenance attestation and bound to the image digest.
7. At deployment, the gate reads the provenance and checks it against policy: the builder must be an approved one, the source repository must be on the allowed list, and the subject digest must match the image actually being deployed.

In practice

Provenance is what defeats the SolarWinds-style attack, where source code was clean but the build process was subverted. If someone builds an image from the correct source on an unauthorised machine, the provenance names that machine, and the gate refuses it.

If this were missing

You could prove an image is unchanged since it was signed, but never prove where it came from or who built it — so a properly signed but illegitimately produced image would sail through.

3.4 SLSA-aligned trusted builds

What it is: SLSA (Supply-chain Levels for Software Artifacts) is an industry framework describing how trustworthy a build process is, expressed as levels. A trusted build is one that meets those requirements: isolated, scripted, and capable of proving what it did.

Why it matters: Signatures and provenance are only as honest as the machine producing them. If a developer can log into the builder mid-run and swap a file, the provenance will faithfully record a lie. Hardening the builder is what makes all the other evidence believable.

The properties a trusted builder must have

Property	What it means in practice
Scripted build	The build is fully defined in a file in version control. Nobody assembles it by hand or clicks buttons to change it.
Isolated	Each build runs in a fresh, throwaway environment that cannot see or influence any other build.
Ephemeral	The environment is destroyed after the build. Nothing persists between runs to carry contamination forward.
No human access	Interactive login to a running build is disabled, so nobody can alter it in flight.
Pinned inputs	Base images and dependencies are referenced by digest, not by a moving tag, so the same build sees the same inputs.
Provenance-generating	The builder itself emits the signed provenance. The thing being described is not the thing writing the description by hand.
Least-privilege identity	The builder holds only the permissions it needs: read the source, push to one registry path, and nothing else.

How it works, end to end

1. A commit lands on a protected branch that requires peer review and blocks direct pushes.
2. The build service starts a fresh, isolated worker with no leftover state from previous runs.

3. The worker proves its identity to the cloud using its instance principal — see section 3.10 — rather than any stored credential.
4. It fetches only the exact commit it was told to build, plus dependencies pinned to exact digests.
5. It executes the build definition from version control. No interactive access is available while it runs.
6. It produces the artifact, the SBOM, and the provenance, and signs them.
7. It pushes everything to the registry using push-only permission scoped to a single repository path.
8. The worker is destroyed. Its short-lived credentials expire and cannot be reused.

If this were missing

The whole chain of evidence rests on sand: a compromised or manually editable builder would produce perfectly valid signatures on malicious artifacts.

3.5 Artifact integrity

What it is: The guarantee that the artifact which arrives is byte-for-byte the artifact that was built, and that all of its evidence still belongs to it.

Why it matters: An artifact passes through several hands between build and runtime — registry storage, replication, caches, download to the host. Integrity is what closes every one of those windows.

How it is achieved, end to end

1. The image is addressed by content, not by name. Its digest is derived from its bytes, so any modification produces a different digest and is immediately visible.
2. The signature covers the digest. Altering the image invalidates the signature; keeping the signature valid requires keeping the image identical.
3. The SBOM and provenance attestations are bound to that same digest, so the evidence cannot be detached and reused on a different image.
4. Deployments reference the digest rather than a tag, removing the risk of a moved nickname pointing somewhere new.
5. The registry stores the image, its signature, and both attestations together under one digest, so evidence travels with the artifact.
6. At pull time, the runtime recomputes the digest of what it downloaded and compares it to what it asked for. A corrupted or substituted download fails the comparison.
7. A scheduled job re-verifies images already running, so an artifact that was valid at admission but has drifted since is detected.

In practice

This is the difference between "the file is called what I expected" and "the file is what I expected." Only the second is a security property.

3.6 Provenance validation

What it is: The act of reading the provenance document at deployment time and deciding, against written policy, whether the origin story is acceptable. Generating provenance is only useful if something actually enforces it.

The checks performed, in order

1. Does a provenance attestation exist for this exact digest? If not, refuse — missing evidence is treated as failed evidence.
2. Is the attestation's signature valid, and does the signing certificate belong to an approved builder identity?
3. Is there a corresponding entry in the transparency log, proving the attestation was published rather than fabricated privately?
4. Does the subject digest inside the document match the digest of the image actually being deployed? This blocks evidence copied from a legitimate image onto a malicious one.
5. Is the source repository on the approved list, and is the branch one that is permitted to reach this environment?
6. Was the build definition the approved workflow, rather than an ad-hoc pipeline someone added?
7. Do the recorded materials — base images and dependencies — meet policy, such as coming only from approved registries?
8. Is the build recent enough, if the environment enforces a maximum artifact age?

Every check must pass. There is no partial credit and no override for convenience: the gate is deliberately fail-closed, so an unreadable or missing document produces a refusal rather than a shrug.

3.7 Deployment policy enforcement

What it is: The automatic checkpoint that sits between "someone asked to deploy this" and "this is running." It re-verifies every piece of evidence from scratch and then admits or blocks.

Why it matters: Evidence that nobody checks is decoration. This gate is the single place where all the earlier work turns into an enforced outcome, and it is the one control that cannot be skipped, because deployment physically passes through it.

How it works, end to end

- 1. A deployment request arrives — a new workload, a rollout, or a scale-up that needs to pull an image.
- 2. The gate intercepts the request before any container is created and extracts the image reference.
- 3. It resolves the reference to an immutable digest, so that everything it evaluates concerns the same exact bytes that will run.
- 4. It fetches the signature, the SBOM attestation, and the provenance attestation for that digest.
- 5. It verifies the signature and confirms the signing identity is on the approved list.
- 6. It confirms the transparency log entry exists and the inclusion proof is valid.
- 7. It validates the provenance against policy, as described in section 3.6.
- 8. It evaluates the SBOM against current vulnerability and licence rules, which may be stricter than they were at build time.
- 9. It applies environment-specific rules — production may require an approved release branch and a higher assurance level than staging.
- 10.If every check passes, the request is admitted and the workload starts. If any check fails, the request is denied, the workload never starts, and an alert is raised with the specific reason.
- 11.The decision, its inputs, and its outcome are written to the immutable audit trail either way.

The rule that makes this work

The gate never trusts an earlier stage's word. It does not ask "did the build system say this was fine?" It fetches the cryptographic evidence and checks it itself, every single time. That is why a compromise of any single earlier stage does not silently propagate.

3.8 OCI IAM compartments

What it is: A compartment in Oracle Cloud Infrastructure is a logical walled area that holds resources. Permissions are written against compartments, so a compartment is the natural boundary of a blast radius.

Why it matters: Without boundaries, one stolen credential reaches everything. With them, it reaches one area, and the damage stops at the wall.

How they were used

Compartment	What lives there	Who can reach it
dev	Developer sandboxes and experimental workloads	Developers, freely
build	Build workers, the registry path they push to	Build identities only, push-scoped

Compartment	What lives there	Who can reach it
staging	Pre-production workloads and test data	Deploy pipeline and QA, read-mostly
prod	Production workloads and customer data	Deploy pipeline via the gate; humans only via break-glass
security	Audit archives, keys, policy definitions, WORM buckets	Security team only; nobody else can write

How it works, end to end

1. Resources are created inside a named compartment rather than in one shared pool.
2. Every policy statement names the compartment it applies to, so a grant in build has no reach into prod.
3. Identities are scoped to the compartment they operate in, and cross-compartment access must be granted deliberately and explicitly.
4. Sensitive material — audit archives, signing policy, keys — lives in the security compartment, which the pipeline can write to but not modify or delete from.
5. Because permissions are compartment-scoped, a compromised build worker cannot read production secrets even though both exist in the same tenancy.

3.9 Dynamic groups

What it is: A group whose membership is decided by a rule rather than a list. Instead of naming individual machines, you describe them: "any compute instance in the build compartment tagged role=builder is a member."

Why it matters: Build infrastructure is created and destroyed constantly. Maintaining a hand-written list of machine identities is impossible to keep accurate, and every stale entry is a permission that outlives the thing it was meant for.

How it works, end to end

1. A rule is written describing the machines that should belong — typically matching on compartment plus a tag such as role=builder.
2. Policies are granted to the group, not to any individual machine.
3. When a new build worker launches with the matching tag, it becomes a member automatically and immediately, with no ticket and no manual step.
4. When that worker is destroyed, it stops being a member automatically. There is no orphaned identity left holding permissions.
5. Because the rule is narrow — a specific compartment and a specific tag — an instance created elsewhere, or without the tag, gains nothing.

Dynamic groups are what make least privilege survive contact with automation. The alternative, in practice, is that someone grants a broad permission once so they never have to touch it again.

3.10 Instance principals

What it is: A mechanism that lets a compute instance authenticate as itself. The machine is the identity. No API key, password, or token file is stored anywhere on it.

Why it matters: Stored credentials are the single most common cause of breaches. They leak into source control, logs, images, backups, and screenshots. A credential that is never stored cannot leak from storage.

How it works, end to end

1. The instance is launched inside a compartment and carries a tag that matches a dynamic group rule.
2. Cloud infrastructure issues the instance a cryptographic identity at boot, tied to the instance itself.
3. Code running on the instance asks the local instance metadata service — reachable only from on the instance — for a token.
4. The metadata service returns a short-lived token that expires in minutes and is automatically refreshed while the instance lives.
5. The instance presents that token when calling cloud services. Its permissions are exactly those granted to the dynamic groups it belongs to.
6. When the instance is terminated, the identity ceases to exist. Nothing needs to be revoked or rotated.

Why this matters more than it sounds

Combine instance principals with keyless signing and the pipeline holds no long-lived secrets at all. There is no key file to exfiltrate, no token to find in a log, and nothing an attacker can steal today and quietly reuse in six months.

3.11 Immutable audit trails

What it is: Records that can be written and read but never edited or deleted. "Immutable" here is enforced by the platform, not by a policy asking people to behave.

Why it matters: The first thing a capable attacker does after acting is clean up the evidence. If logs can be edited by whoever holds administrator rights, they prove nothing about a serious incident — which is exactly when you need them.

The four layers of record kept

Layer	What it records	Why it cannot be tampered with
Rekor transparency log	Every signature and attestation, with a timestamp	Append-only and cryptographically chained; removing an entry breaks the maths
OCI Audit service	Every API call: who, what, when, source IP, allowed or denied	Written by the platform, outside the caller's control, retained by policy
Policy gate decision log	Every admit and block decision with the reason and evidence	Streamed out immediately to storage the pipeline cannot rewrite
WORM object storage	Archived SBOMs, provenance documents, and decisions	Retention lock; deletion is refused until the retention period ends, even for administrators

How it works, end to end

1. Each component writes its records to a destination it can add to but not modify.
2. Records are written as events happen, not batched up and summarised later where detail is lost.
3. Archive buckets carry a retention lock, so an early delete request is refused by the storage service itself.
4. Records stream into monitoring, where alarms fire on meaningful patterns — a spike in blocked deployments, an edit to a policy, an unusual identity pushing to the registry.
5. Because signatures are also in a public transparency log, an attacker cannot quietly un-publish a signature they created.

3.12 Least privilege across 20+ integrations

What it is: The systematic reduction of permissions across every system the pipeline touches — source control, build service, registries, secret stores, databases, queues, monitoring, ticketing, storage, and more — so each holds only what it genuinely needs.

Why it matters: Integrations accumulate quietly. Each one is set up in a hurry with broad access "just to get it working," and nobody revisits it. Twenty such integrations mean twenty possible entry points, most with far more access than their function requires.

The method applied to each integration

1. Inventory: list every integration, what it connects to, and what credential it currently uses.
2. Observe: examine audit logs to see which permissions are actually exercised, rather than which were requested.

3. Right-size: write a new policy granting only the observed, justified actions, scoped to a single compartment and resource type.
4. Replace credentials: swap static keys for instance principals or short-lived tokens wherever the integration supports it.
5. Separate duties: split identities so the one that can push cannot deploy, and the one that can deploy cannot rewrite audit records.
6. Test: run the integration against the narrowed policy in a lower environment and confirm nothing legitimate breaks.
7. Roll out and watch: apply to production and monitor denials closely for a period, since a denial here means either an attack or a permission genuinely still needed.
8. Review on a schedule: permissions drift upward over time unless someone deliberately pulls them back.

The separation that matters most

The build identity can push to the registry but cannot deploy. The deploy identity can pull and run but cannot push. Neither can alter the audit trail or the policy that governs them. So compromising any one identity is not enough to both introduce a malicious artifact and get it running.

4. The End-to-End Workflow

This section follows one container image through its entire life: from a developer typing git push to that image running in production, and then being re-checked afterwards. The step numbers match the numbered boxes in the workflow diagram at the end of this section.

4.1 Stage one — writing and accepting the code (step 1)

Step 1 — Developer pushes a signed, peer-reviewed commit. A developer finishes a change and opens a pull request. The branch is protected: nobody, including administrators, can push directly to it. At least one other engineer must review and approve. Commits are signed, so the repository records who wrote each change. Once merged, the commit hash becomes the anchor for every later piece of evidence — everything that follows can be traced back to this exact revision.

4.2 Stage two — the trusted build (steps 2 to 5)

Step 2 — The build worker proves who it is. The merge triggers a build. A fresh worker starts and needs permission to fetch source, read dependencies, and later push. It does not read a stored key. It asks the local metadata service for a short-lived token, which it can only obtain because it is that instance, in that compartment, with that tag. Its permissions come from the dynamic group it automatically belongs to.

Step 3 — Building in a clean, isolated workspace. The worker checks out the exact commit and nothing else. Dependencies and base images are pinned to exact digests rather than moving tags, so the build cannot silently pick up a different version than the one that was reviewed. The environment is brand new and will be destroyed afterwards, and interactive login is disabled for the duration.

Step 4 — Generating the SBOM. The finished image is scanned and every package inside it is inventoried with name, version, and licence. The scan reads the built image rather than the dependency file, so it captures what actually ended up inside, including transitive dependencies nobody asked for directly.

Step 5 — Generating in-toto provenance. The builder writes the birth certificate: the source repository and commit, its own builder identity, the workflow and parameters used, the digests of the materials it consumed, the digest of the artifact it produced, and the start and finish times.

4.3 Stage three — signing and logging (steps 6 to 8)

Step 6 — Obtaining a short-lived certificate. cosign requests an identity token from the CI system describing this specific workflow run, and exchanges it at Fulcio for a certificate valid for about ten minutes. The certificate permanently records which workflow and repository it was issued to.

Step 7 — Signing the digest. cosign generates a key in memory, signs the image digest plus both attestations, and discards the key. The certificate then expires. Nothing reusable is left behind.

Step 8 — Publishing to the transparency log. The signature and certificate are entered into Rekor, which timestamps them and returns an inclusion proof. This creates a public, append-only record that this identity signed this digest at this moment — one that the signer cannot later erase.

4.4 Stage four — storing artifact and evidence together (steps 9 and 10)

Step 9 — Pushing to the registry. The worker pushes four linked objects to OCIR: the image itself, its signature, the SBOM attestation, and the provenance attestation. Its credentials allow push to one repository path in one compartment and nothing more — it cannot delete, cannot overwrite other repositories, and cannot deploy.

Step 10 — Everything is bound to one digest. All four objects are stored against the same immutable digest. From this point the artifact travels with its own evidence: anyone who receives the image can independently obtain and check its proofs without asking the build system anything.

4.5 Stage five — the policy gate (steps 11 to 13)

Step 11 — The deployment request is held. Someone or something requests a deployment. The gate intercepts the request before any container is created, resolves the image reference to its digest, and fetches the signature and both attestations.

Step 12 — Re-verifying everything from scratch. The gate checks the signature mathematically; confirms the signing identity is an approved workflow; confirms the Rekor entry exists with a valid inclusion proof; validates the provenance against the approved builder, source repository, and branch; confirms the provenance subject digest matches the image being deployed; and evaluates the SBOM against today's vulnerability and licence rules.

Step 13 — Admit or block. If every check passes, the deployment is admitted. If any check fails, it is blocked outright, the workload never starts, and an alert names the specific failed check. The gate is fail-closed: missing or unreadable evidence is treated exactly like invalid evidence.

What a blocked deployment looks like

- No signature found for the digest — likely an image built outside the pipeline.
- Signature valid but the identity is unapproved — someone signed from a personal account or an unapproved workflow.
- No transparency log entry — the signature was produced privately and never published.
- Provenance names an unapproved builder or source repository.
- Provenance subject digest does not match the deployed image — evidence copied from another artifact.
- SBOM contains a package with a critical vulnerability above the environment's threshold.
- SBOM contains a licence not permitted for a distributed product.

4.6 Stage six — runtime and continuous assurance (steps 14 to 16)

Step 14 — The workload runs with its own minimal identity. The container starts under an identity scoped to a single compartment, holding only the permissions its function requires. It does not inherit the build identity's permissions, so an application vulnerability does not hand an attacker the ability to push images.

Step 15 — Everything is recorded permanently. Every API call the workload makes is captured in the audit service with actor, action, time, source, and outcome. The SBOM, provenance, and gate decision are archived to retention-locked storage, so the full evidence package for this deployment can be produced years later.

Step 16 — Continuous re-verification and drift detection. A scheduled job re-checks what is actually running: are the signatures still valid, do the digests still match what was admitted, and have new vulnerabilities been disclosed against packages listed in the SBOM? Anything that has drifted since admission is flagged, because a deployment that was safe on Tuesday is not automatically safe on Friday.

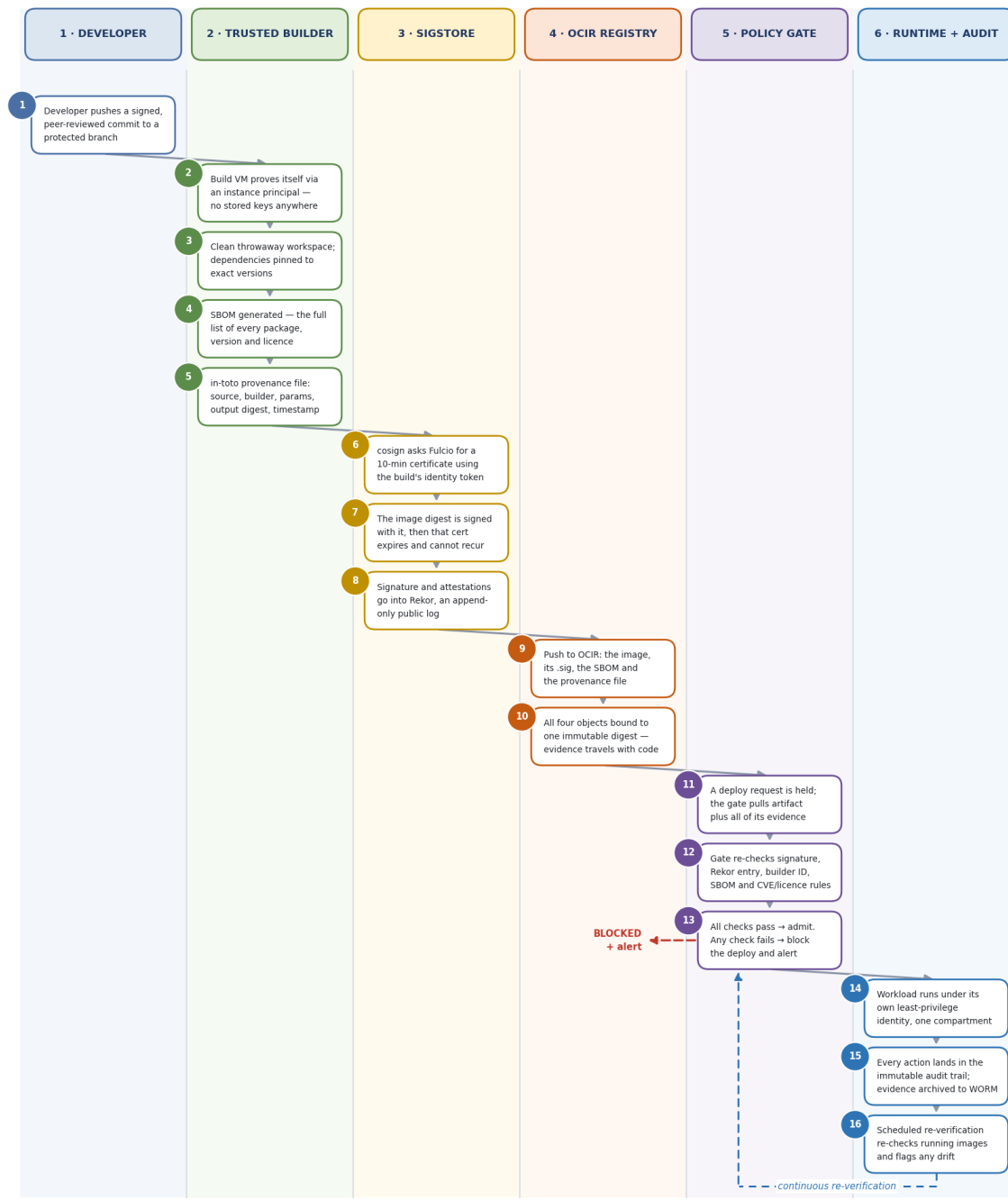
4.7 The workflow at a glance

The diagram below shows the same 16 steps as columns of responsibility. Read it top to bottom; each step must succeed before the next begins.

Diagram 1 — End-to-End Workflow

End-to-End Workflow: One Container Image, From Commit to Running Pod

Follow the numbers. Each step must succeed before the next one is allowed to begin.



Fail-closed: if any proof is missing, unreadable or unsigned, the answer is NO — the deploy stops rather than continues.

Steps 1-10 create the evidence. Steps 11-13 check the evidence. Steps 14-16 keep checking it after go-live.

Diagram 1: The 16 steps of the pipeline, grouped by which system performs them.

5. The Architecture Diagram Explained

The workflow diagram showed the order things happen in. This architecture diagram shows how the system is arranged: three zones working together, with the pipeline in the middle, identity on the left, and evidence on the right.

Every zone, box, and arrow is explained after the figure.

5.1 How to read the diagram in thirty seconds

The tall column down the middle is the journey of the software, top to bottom: source code becomes a build, the build is signed, the signed result is stored, the stored result is checked, and only then does it run.

The panel on the left is identity: who or what is allowed to do each of those things. The panel on the right is evidence: the permanent record that each of those things happened. Solid arrows show the software moving. Dashed arrows on the left show permission being granted; dashed arrows on the right show evidence being written.

Diagram 2 — System Architecture

Secure Software Supply Chain + Least-Privilege Access on OCI

End-to-end architecture: nothing is built, stored, or deployed without proof of who made it and how

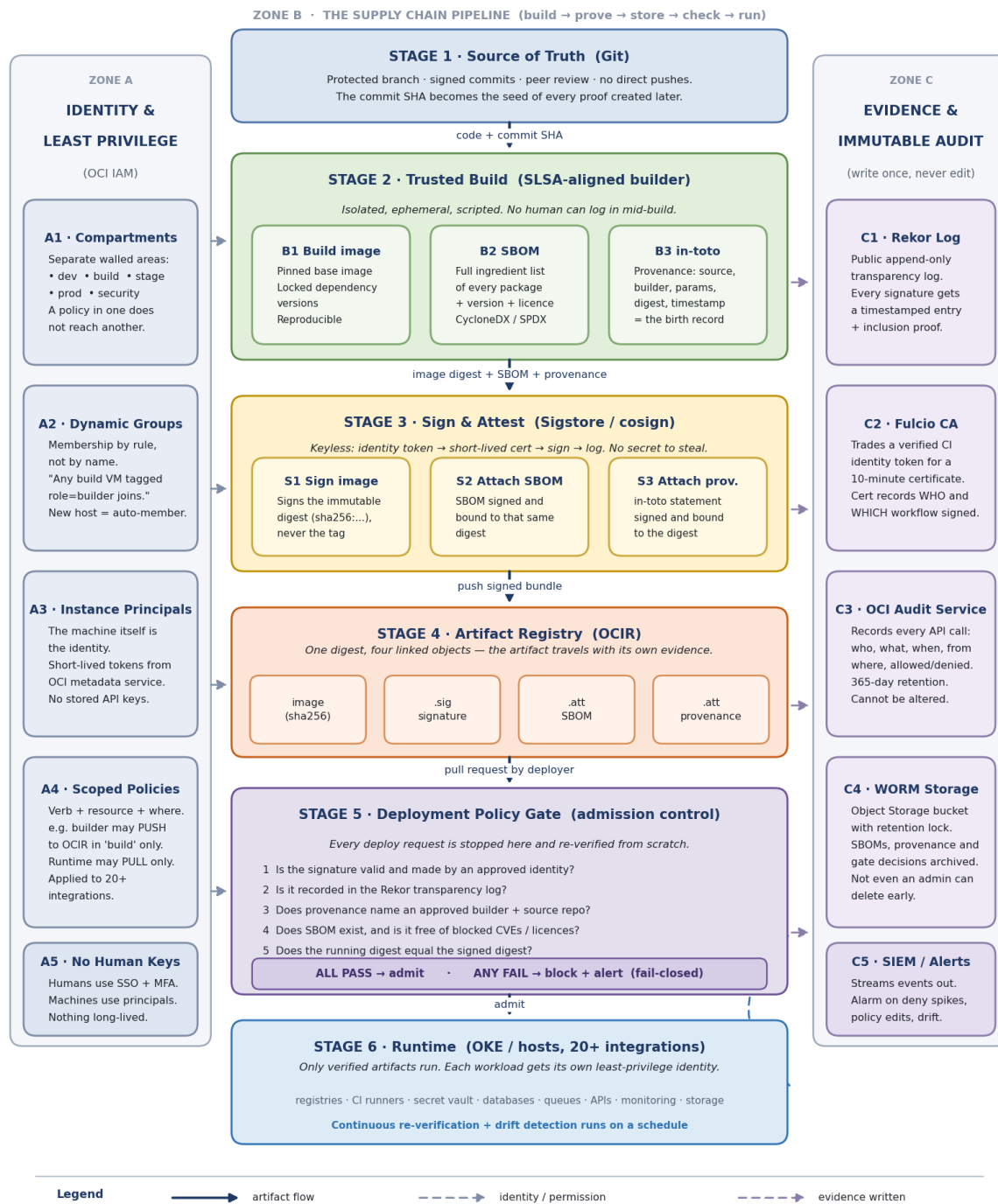


Diagram 2: Zone A (identity), Zone B (the pipeline), Zone C (evidence and audit).

5.2 Zone A (left panel) — identity and least privilege

This panel represents Oracle Cloud IAM. Nothing in the centre column can act without first obtaining an identity from here.

Box	What it is	What it does in this system
A1 · Compartments	Walled areas holding resources	Separates dev, build, staging, prod, and security so a permission in one cannot reach another
A2 · Dynamic Groups	Groups defined by a rule, not a list	Any build machine with the right tag in the right compartment joins automatically; destroyed machines leave automatically
A3 · Instance Principals	The machine as its own identity	Build workers get short-lived tokens from the metadata service instead of storing API keys
A4 · Scoped Policies	Narrow permission statements	The builder may push to one registry path in the build compartment; the runtime may only pull. Applied across 20+ integrations
A5 · No Human Keys	Credential hygiene rule	People authenticate with single sign-on and MFA; machines use principals. Nothing long-lived exists to steal

5.3 Zone B (centre column) — the supply chain pipeline

Stage 1 — Source of truth

The Git repository. Protected branches, signed commits, mandatory review, and no direct pushes. The commit hash produced here is the seed from which every later proof grows.

Stage 2 — Trusted build

The SLSA-aligned builder: isolated, ephemeral, fully scripted, with no interactive human access. It contains three sub-boxes:

- B1 Build image — compiles and packages using a pinned base image and locked dependency versions, so the build is reproducible.
- B2 SBOM — produces the complete ingredient list of every package, version, and licence in CycloneDX or SPDX format.
- B3 in-toto — produces the provenance document naming the source, builder, parameters, output digest, and timestamp.

Stage 3 — Sign and attest

Sigstore and cosign, operating keylessly: an identity token is exchanged for a short-lived certificate, the signing happens, and the certificate expires. Three sub-boxes:

- S1 Sign image — signs the immutable digest, never the tag, so moving a tag cannot carry a signature with it.
- S2 Attach SBOM — signs the SBOM and binds it to that same digest.
- S3 Attach provenance — signs the in-toto statement and binds it to that same digest.

Stage 4 — Artifact registry

OCIR holds four linked objects under one digest: the image, its .sig signature, the SBOM attestation, and the provenance attestation. This is what is meant by the artifact travelling with its own evidence — a consumer can verify it without contacting the build system.

Stage 5 — Deployment policy gate

The admission checkpoint. It runs five checks in sequence, shown as a numbered list inside the box:

- Is the signature valid, and made by an approved identity?
- Is it recorded in the Rekor transparency log?
- Does the provenance name an approved builder and source repository?
- Does an SBOM exist, and is it free of blocked vulnerabilities and licences?
- Does the digest being deployed equal the digest that was signed?

The shaded bar at the bottom of the box states the verdict rule: all checks pass means admit; any check fails means block and alert. It is fail-closed by design.

Stage 6 — Runtime

The cluster and hosts where verified workloads run, spanning the 20+ integrated systems listed in the box — registries, CI runners, secret vault, databases, queues, APIs, monitoring, and storage. Each workload receives its own least-privilege identity. The blue dashed arrow curving from this stage back up to Stage 5 represents scheduled re-verification and drift detection: running workloads are re-checked continuously, not trusted forever because they passed once.

5.4 Zone C (right panel) — evidence and immutable audit

Box	What it is	What it guarantees
C1 · Rekor Log	Public append-only transparency log	Every signature gets a timestamped entry and inclusion proof that the signer cannot later remove
C2 · Fulcio CA	Certificate authority for keyless signing	Exchanges a verified CI identity token for a ten-minute certificate that records who and which workflow signed
C3 · OCI Audit Service	Platform-level API call recording	Captures who, what, when, from where, and allowed or denied, outside the caller's control

Box	What it is	What it guarantees
C4 · WORM Storage	Retention-locked object storage	Archives SBOMs, provenance, and gate decisions where even an administrator cannot delete them early
C5 · SIEM / Alerts	Monitoring and alerting	Streams events out and raises alarms on denial spikes, policy edits, and drift

5.5 The arrows, explained

Arrow	Meaning
Solid vertical arrows in the centre	The artifact moving forward: code and commit hash, then image plus SBOM plus provenance, then the signed bundle pushed to the registry, then the pull request by the deployer, then admission into runtime
Dashed arrows from the left panel	Identity and permission being granted into a pipeline stage — the builder receiving a token, the signer receiving an identity, push and pull rights being scoped, the gate receiving its own identity
Dashed arrows to the right panel	Evidence being written outward — the build record, the certificate and log entry, the archived evidence copy, and the gate's logged decision
Blue dashed curve from Stage 6 to Stage 5	Continuous re-verification: running workloads are re-checked on a schedule and drift is flagged
Red dashed arrow (workflow diagram)	The blocked path — a deployment that fails any check is stopped and an alert is raised

5.6 The one rule the whole design rests on

Trust rule

A later stage never trusts an earlier stage's word. It re-checks the cryptographic proof itself.

This is why the architecture is drawn as evidence flowing outward to an immutable record rather than as approvals flowing forward. If any single stage is compromised, the compromise does not silently propagate — the next stage still demands proof that the compromised stage cannot forge.

6. Which Attack Each Piece Stops

A useful way to sanity-check a security design is to name the specific attack each control defeats. If a control does not map to a real attack, it is probably ceremony.

Attack	What the attacker does	What stops it
Poisoned registry image	Steals registry credentials and pushes a malicious image under a trusted name	Signature verification at the gate — the impostor has no valid signature from an approved identity
Tag repointing	Moves the :latest tag to point at a malicious image	Signatures bind to digests, and deployments reference digests, so the moved nickname carries no proof
Compromised build server	Subverts the build so clean source produces a malicious artifact	in-toto provenance plus SLSA builder hardening — the unapproved builder is named in the provenance and rejected
Malicious dependency	Publishes a backdoored version of a popular library	SBOM plus vulnerability scanning at build and again at deploy, with pinned dependency digests
Evidence transplant	Copies valid attestations from a legitimate image onto a malicious one	Subject digest matching — the provenance names a different digest than the one being deployed
Private signing	Signs an artifact but never publishes, hoping nobody notices	Transparency log check — no Rekor entry means no admission
Stolen signing key	Exfiltrates a long-lived private key and signs at will	Keyless signing — the key exists in memory for seconds and there is nothing persistent to steal
Stolen static credential	Finds an API key in a log, image layer, or repository	Instance principals — machines hold no stored credentials, only short-lived tokens
Lateral movement	Uses one compromised service account to reach unrelated systems	Compartments plus scoped policies — permission in one compartment grants nothing in another
Stale machine identity	Uses an identity left behind by a decommissioned host	Dynamic groups — membership follows the rule, so a destroyed instance loses access automatically
Log tampering	Deletes or edits records to hide activity	Append-only transparency log, platform-written audit records, and retention-locked WORM archives

Attack	What the attacker does	What stops it
Environment promotion abuse	Pushes an image built for test straight into production	Environment-specific policy — production requires an approved release branch and higher assurance

7. Explaining This Project in One Minute

If you need to describe this project out loud, the version below covers it without jargon.

The spoken version

"Most software today is assembled rather than written — most of what ships is code from other people, put together by automated build servers. That creates a lot of places where something harmful can be inserted without anyone touching our source code.

So we stopped trusting the pipeline and started requiring proof. Every image we build is cryptographically signed, ships with a complete list of everything inside it, and carries a signed record of exactly which source code and which build machine produced it. Then we put a gate in front of deployment that re-checks all of that from scratch and blocks anything that fails — no exceptions, and missing evidence counts as a failure.

Alongside that, we cut permissions right back across more than twenty integrated systems. Machines authenticate as themselves with short-lived tokens instead of stored keys, permissions are scoped to one walled-off compartment at a time, and the identity that can build cannot deploy while the identity that can deploy cannot build.

Everything either half does is written to records that nobody can edit or delete — so if we ever have to answer 'what exactly was running, where did it come from, and who allowed it,' we can prove the answer rather than assert it."

7.1 The three sentences worth remembering

1. Trust was replaced with proof: every artifact carries cryptographic evidence of what it is and where it came from.
2. Every stage re-verifies that evidence independently, and refuses by default when anything is missing.
3. Every identity holds the minimum access it needs, holds it briefly, and leaves a permanent record of using it.